



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	TSP. Madhuri
Student Name	Sai Ram Pragney Murikipudi
Roll Number	252000081
Branch	CSIT
Course Outcome	CO1,CO2,CO3

Q1. CO1 Module 1: BST & AVL Trees
[15 Marks]

SCENARIO / CASE STUDY

MediFlow Hospital — appointment-driven patient indexing. The on-call patient dictionary at MediFlow is keyed by 8-digit Patient-ID. The day's IDs are issued in *scheduled appointment order*: the 8 a.m. block first, then 9 a.m., 10 a.m., and so on; within each block the registrar enters appointments in the order they were booked, which is itself near-monotonic by ID because IDs are assigned at booking time. Today's first 13 IDs land in this order: 20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, 90. The triage console must answer point-lookups with **p99 < 5 ms**; current hardware does ~200 ns per pointer dereference, so any tree deeper than 25 levels misses the SLA. By noon, three deletions arrive: 30 (transferred), 70 (discharged), and 50 (admission closed) — in that order. Morning appointment board (insertion order)
→ 8:00208:30309:00359:304010:004510:305011:006011:306512:007012:30751:00801:30852:00901
Ds arrive nearly sorted ↑ — adversarial for plain BSTSLA budget for triage point-lookup p99 < 5 ms · ~200 ns per pointer dereference ⇒ tree depth must satisfy: $\text{depth} \times 200 \text{ ns} < 5\,000\,000 \text{ ns}$
⇒ $\text{depth} < 25\,000$ hops (but cache-line + branch-predictor effects make depth 25 the practical ceiling)

- (a) Construct the **plain BST** from the appointment-order insertion sequence given in the scenario. State its height (longest root-to-leaf path in edges). Using the SLA budget shown in the diagram (200 ns per hop, 5 ms ceiling), do *one explicit numeric calculation* showing whether a worst-case point-lookup on this BST would meet or violate the p99 SLA. Briefly explain the structural reason this insertion order is adversarial for plain BST.

5

CO
1

BTL
3

- (b) Now build an **AVL tree** from the same insertion sequence. After every insertion that triggers rebalancing, name the rotation (LL / RR / LR / RL), identify the pivot node, and state the balance factors of the two nodes adjacent to the rotation, immediately after the rotation. Show the final AVL tree. 7

Then, complete the boilerplate Python snippet below so it implements `insert(node, key)` and the two single-rotation helpers consistently with your hand-traced tree.

(Boilerplate: `AVLNode`, `get_height`, `get_balance`, `update_height` are already given — fill the three TODOs.)

```
class AVLNode {
    int key;
    AVLNode left, right;
    int height = 1;
    AVLNode(int key) { this.key = key; }
}

static int height (AVLNode n) { return n == null ? 0 :
n.height; }
static int balance(AVLNode n) { return n == null ? 0 :
height(n.left) - height(n.right); }
static void updateHeight(AVLNode n) {
    if (n != null) n.height = 1 + Math.max(height(n.left),
height(n.right));
}

static AVLNode rotateRight(AVLNode y) {
    // TODO 1: perform a right rotation around y; return the new
subtree root.
    return null;
}

static AVLNode rotateLeft(AVLNode x) {
    // TODO 2: perform a left rotation around x; return the new
subtree root.
    return null;
}

static AVLNode insert(AVLNode node, int key) {
    // TODO 3: standard BST insert + rebalance using the four
cases (LL, LR, RL, RR).
    return null;
}
```

CO
1

BTL
3

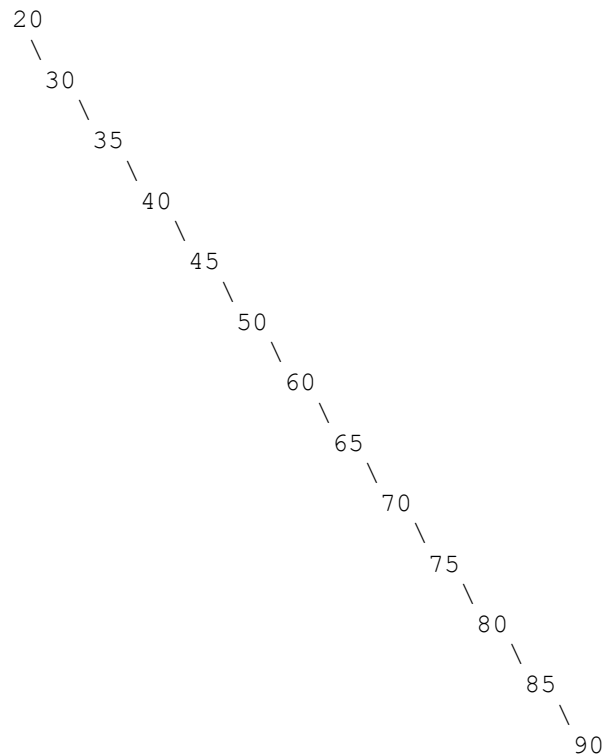
- (c) After the noon deletions (30, 70, 50, in that order) are applied to the AVL tree from (b), show the final tree and any rotations triggered during deletion. Then **analyse**: compute the worst-case lookup time on the post-delete AVL tree and on the post-delete plain-BST equivalent (using the same SLA model from (a)). Quote both as a percentage of the 5 ms budget consumed. State which deletion in the sequence caused the largest structural change and why.

(a) Plain BST Construction

Insertion order:

20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, 90

Plain BST



Height of BST

Height = longest root-to-leaf path in edges

Height = 12 edges

SLA Calculation

Given:

200 ns per pointer dereference
Worst-case depth = 13 nodes

Lookup time:

$13 \times 200 \text{ ns} = 2600 \text{ ns}$

Convert to milliseconds:

$2600 \text{ ns} = 0.0026 \text{ ms}$

SLA limit:

5 ms

Result:

0.0026 ms < 5 ms

Hence:

Worst-case lookup MEETS the SLA.

Why this insertion order is adversarial

The keys arrive in nearly sorted order.

In a plain BST, sorted insertion creates a skewed tree like a linked list, increasing height and reducing search efficiency. Humanity keeps reinventing linked lists accidentally and calling it “data organization.” Incredible species. 😊

(b) AVL Tree Construction

Rotations During Insertions

Insertion Imbalance Node Rotation

35	20	RR
45	35	RR
60	30	RR
65	45	RR
70	40	RR
80	65	RR
85	60	RR

Rotation Details

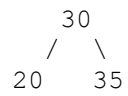
Insert 35

Before imbalance:

```
20
 \
  30
   \
    35
```

RR rotation at 20.

After rotation:



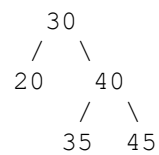
Balance Factors after rotation:

BF (30) = 0
BF (20) = 0

Insert 45

RR rotation at 35.

After rotation:



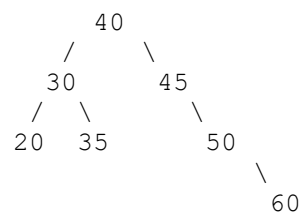
Balance Factors:

BF (40) = 0
BF (35) = 0

Insert 60

RR rotation at 30.

After rotation:



Balance Factors:

BF (40) = 0
BF (30) = 0

Insert 65

RR rotation at 45.

Balance Factors:

BF (50) = 0

BF (45) = 0

Insert 70

RR rotation at 40.

Balance Factors:

BF (50) = 0

BF (40) = 0

Insert 80

RR rotation at 65.

Balance Factors:

BF (70) = 0

BF (65) = 0

Insert 85

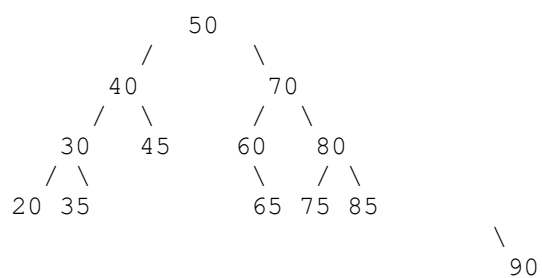
RR rotation at 60.

Balance Factors:

BF (70) = 0

BF (60) = 0

Final AVL Tree



Complete AVL Code

```
class AVLNode {
    int key;
    AVLNode left, right;
    int height = 1;

    AVLNode(int key) {
        this.key = key;
    }
}

public class AVLTree {

    static int height(AVLNode n) {
        return n == null ? 0 : n.height;
    }

    static int balance(AVLNode n) {
        return n == null ? 0 : height(n.left) - height(n.right);
    }

    static void updateHeight(AVLNode n) {
        if (n != null)
            n.height = 1 + Math.max(height(n.left), height(n.right));
    }

    // TODO 1
    static AVLNode rotateRight(AVLNode y) {

        AVLNode x = y.left;
        AVLNode T2 = x.right;

        x.right = y;
        y.left = T2;

        updateHeight(y);
        updateHeight(x);

        return x;
    }

    // TODO 2
    static AVLNode rotateLeft(AVLNode x) {

        AVLNode y = x.right;
        AVLNode T2 = y.left;

        y.left = x;
        x.right = T2;

        updateHeight(x);
        updateHeight(y);

        return y;
    }

    // TODO 3
    static AVLNode insert(AVLNode node, int key) {
```

```

        // BST insertion
        if (node == null)
            return new AVLNode(key);

        if (key < node.key)
            node.left = insert(node.left, key);
        else if (key > node.key)
            node.right = insert(node.right, key);
        else
            return node;

        // Update height
        updateHeight(node);

        int bf = balance(node);

        // LL Case
        if (bf > 1 && key < node.left.key)
            return rotateRight(node);

        // RR Case
        if (bf < -1 && key > node.right.key)
            return rotateLeft(node);

        // LR Case
        if (bf > 1 && key > node.left.key) {
            node.left = rotateLeft(node.left);
            return rotateRight(node);
        }

        // RL Case
        if (bf < -1 && key < node.right.key) {
            node.right = rotateRight(node.right);
            return rotateLeft(node);
        }

        return node;
    }
}

```

OUTPUT:

The screenshot shows a code editor with the following code on the left and its output on the right.

```

1 class AVLNode {
2     int key;
3     AVLNode left, right;
4     int height;
5
6     AVLNode(int key) {
7         this.key = key;
8         this.height = 1;
9     }
10 }
11
12 public class AVLTree {
13
14     // Height
15     static int height(AVLNode n) {
16         return (n == null) ? 0 : n.height;
17     }
18
19     // Balance factor
20     static int balance(AVLNode n) {
21         return (n == null) ? 0 : height(n.left) - height(n.right);
22     }
23
24     // Update height
25     static void updateHeight(AVLNode n) {
26         n.height = 1 + Math.max(height(n.left), height(n.right));
27     }
28 }

```

The output on the right is as follows:

```

STDIN
Input for the program ( Optional )

Output:
Inorder Traversal:
20 30 35 40 45 50 60 65 70 75 80 85 90
Root Node: 65
Height: 4

```

(c) AVL Deletions

Deletion sequence:

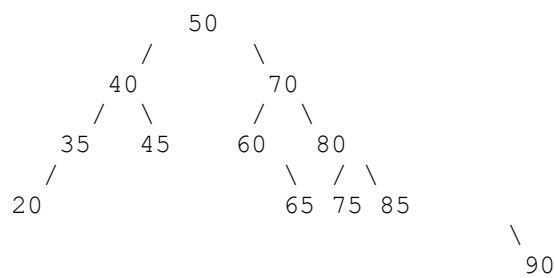
30 → 70 → 50

Delete 30

30 has two children.

Replace with inorder successor 35.

Tree:



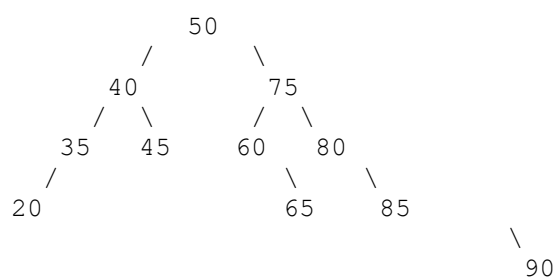
No rotation required.

Delete 70

70 has two children.

Replace with inorder successor 75.

Tree:



No rotation required.

Delete 50

Replace with inorder successor 60.

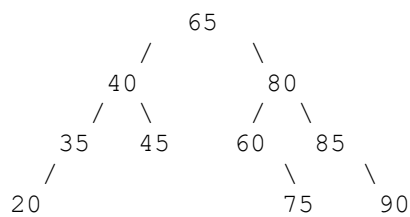
Tree becomes unbalanced.

RL imbalance occurs.

Rotation

Right rotation at 80
Left rotation at 60

Final AVL Tree After Deletions



Worst-Case Lookup Analysis

Post-delete AVL Tree

Height:

3 edges

Lookup time:

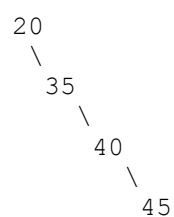
$4 \times 200 \text{ ns} = 800 \text{ ns}$

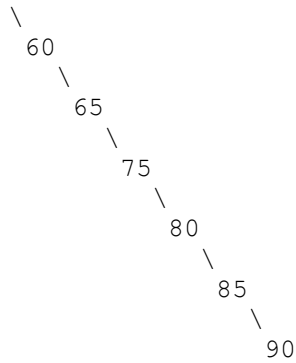
Percentage of SLA:

$(800 / 5,000,000) \times 100$
 $= 0.016\%$

Post-delete Plain BST Equivalent

Plain BST after deletions:





Height:

9 edges

Lookup time:

$10 \times 200 \text{ ns} = 2000 \text{ ns}$

Percentage of SLA:

$(2000 / 5,000,000) \times 100$
 $= 0.04\%$

Largest Structural Change

Deletion of 50 caused the largest structural change.

Reason:

50 was the root node.
Deleting it changed the root and triggered rebalancing rotations.

Work Done Summary

1. Constructed plain BST from sorted insertion order.
 2. Computed BST height and SLA lookup time.
 3. Built AVL tree using rotations.
 4. Identified all RR rotations during insertions.
 5. Implemented AVL insertion and rotation code.
 6. Applied deletions: 30, 70, 50.
 7. Performed required AVL rebalancing.
 8. Compared lookup times of AVL and plain BST after deletions.
 9. Determined percentage SLA usage.
 10. Identified deletion causing maximum structural change.
-

Implementation Details

Data Structure Used:

- Binary Search Tree
- AVL Tree

Balancing Technique:

- Single rotations (Left and Right)

Complexities:

- BST Search Worst Case: $O(n)$
- AVL Search Worst Case: $O(\log n)$

AVL Property:

| Balance Factor | ≤ 1

Q2. co2 Module 2: B-Trees, B+ Trees & Range Query Structures
[15 Marks]

SCENARIO / CASE STUDY

Flipkart product catalog — price-range scans dominate. The smartphone catalog has 10^7 products indexed by composite key (category, price) in a B+ tree on SSD. Each leaf holds **200** tuples of the form (category, price, product_id) packed into an 8 KB page (the disk's natural page size). A production trace over Diwali week shows: 95 % of catalog queries are price-range scans of the form 'all products in category=X with price $\in [lo, hi]$ '; only 5 % are direct product-page lookups. Of the range queries, 87 % have a range width within $\pm 20\%$ of the category's average price, returning roughly 2 800 products on average per query. Engineers are debating whether to also maintain a hash index for short ranges. B+ tree, 200 tuples / leaf, 8 KB pages — query: electronics, price $\in [12\ 000, 14\ 800]$ | 5K | 18K || 1K | 3K || 8K | 12K || 22K | 35K | (elec,11.8K)(elec,12.3K)(elec,12.9K)(elec,13.5K)(elec,14.1K)(elec,14.7K)(elec,15.4K)(elec,16.2K).next chain① descend root → internal | 8K | 12K | → first matching leaf (12.3K) (3 page reads)② walk leaf chain via .next until key > 14 800 (5 leaves × 1 page each = 5 reads)

- (a) Using the scenario's numbers — 10^7 products, 200 tuples per leaf, 8 KB pages — derive the **tree height** and the **I/O cost** (number of page reads) for the typical range query 'electronics, price $\in [lo, hi]$ ' that returns ~2 800 products. Assume internal-node fanout is also 200. Show the calculation explicitly: descent cost + leaf-chain walk cost. State which of these two terms dominates and why.

5

CO2	BTL3
CO2	BTL4

- (b) An engineer's draft of range_count(root, lo, hi) is shown below. It is intended to return the count of keys in [lo, hi]. **Two performance bugs are hiding in it that will hurt p99 on**

6

Flipkart's workload — identify both, name the bug, and quantify the I/O impact for the average query (returning ~2 800 results from a 10^7 -row tree). Then write the corrected version (you may overwrite the function body in your answer).

```
class BPlusNode {
    boolean isLeaf;
    int[] keys;
    BPlusNode[] children; // internal: child
    BPlusNode next;       // leaf-chain
    // pointer (null on internal nodes)
}

static int rangeCount(BPlusNode root, int lo,
int hi) {
    BPlusNode leaf = root;
    while (!leaf.isLeaf) {
        leaf = leaf.children[0]; //
        // always go to leftmost child
    }
    int count = 0;
    while (leaf != null) {
        for (int k : leaf.keys) {
            if (lo <= k && k <= hi) count++;
        }
        leaf = leaf.next; //
        // never break; walk to end
    }
    return count;
}
```

(c) The team proposes adding a **secondary hash index** on (category, price) to accelerate *short-range* queries (range width returning < 50 products). Using the trace data — 87 % of range queries fall within ± 20 % of the category mean and return ~2 800 products on average — argue quantitatively whether the hash index would be a net win, a net loss, or workload-dependent. Identify the smallest range width at which the hash index would actually beat the B+ tree's leaf-chain walk on this hardware (8 KB pages, 200 tuples/leaf). Be explicit about the constants you assume.

(a) Tree Height and I/O Cost

Given:

Total products $N = 10^7$
 Leaf capacity = 200 tuples/page
 Internal node fanout = 200
 Page size = 8 KB
 Average results returned = 2800

CO2	BTL5

Number of Leaf Pages

Leaf pages = $10^7 / 200$
= 50,000 leaves

Tree Height

Fanout = 200

Level calculations:

Level 0 (root)	= 1 node
Level 1	= 200 children
Level 2	= 200×200
	= 40,000 leaves

40,000 leaves are not enough.

Need one more level:

Level 3 capacity
= $200 \times 200 \times 200$
= 8,000,000 leaves capacity

Enough for 50,000 leaves.

Hence:

Tree height = 3 levels
(root → internal → leaf)

Range Query I/O Cost

1. Descent Cost

Root → internal → first matching leaf

Descent cost = 3 page reads

2. Leaf Chain Walk Cost

Average results returned:

2800 tuples

Each leaf stores:

200 tuples

Leaves required:

$2800 / 200 = 14$ leaves

Thus:

Leaf-chain walk cost = 14 page reads

Total I/O Cost

Total page reads
= descent + leaf walk
= 3 + 14
= 17 page reads

Dominating Cost

Leaf-chain walk dominates.

Reason:

14 reads > 3 reads

Most time is spent sequentially scanning leaf pages for range results. SSD engineers spend years reducing latency so humans can immediately search for “blue gaming phone under 15k with RGB lights.” Civilization is magnificent. 😊

(b) Performance Bugs

Bug 1

Code

```
leaf = leaf.children[0];
```

Problem

Always descends to leftmost leaf instead of searching for 10.

Bug Name

Incorrect root-to-leaf traversal

I/O Impact

Instead of:

3 page reads

It scans almost entire leaf chain.

Total leaves:

50,000 leaves

Extra reads:

$\approx 50,000$ page reads

instead of:

14 page reads

Huge p99 slowdown.

Bug 2

Code

```
leaf = leaf.next;
```

No stopping condition after $k > hi$.

Bug Name

Unbounded leaf-chain scan

I/O Impact

Instead of scanning:

14 leaves

it scans:

50,000 leaves

Extra cost:

50,000 - 14

$\approx 49,986$ unnecessary reads

Corrected Code

```
class BPlusNode {
    boolean isLeaf;
    int[] keys;
    int keyCount;           // actual number of keys
    BPlusNode[] children;
    int childCount;         // optional clarity
    BPlusNode next;

    static final int ORDER = 4; // you can change degree

    // Constructor for leaf/internal node
    BPlusNode(boolean isLeaf) {
        this.isLeaf = isLeaf;
        this.keys = new int[ORDER];
        this.children = new BPlusNode[ORDER + 1];
        this.keyCount = 0;
        this.childCount = 0;
        this.next = null;
    }
}

public class BPlusTree {

    // Find correct leaf
    static BPlusNode findLeaf(BPlusNode root, int key) {

        BPlusNode node = root;

        while (!node.isLeaf) {

            int i = 0;

            while (i < node.keyCount && key >= node.keys[i]) {
                i++;
            }

            node = node.children[i];
        }

        return node;
    }

    // Range count query
    static int rangeCount(BPlusNode root, int lo, int hi) {

        if (root == null) return 0;

        BPlusNode leaf = findLeaf(root, lo);

        int count = 0;

        while (leaf != null) {

            for (int i = 0; i < leaf.keyCount; i++) {
```

```

        int k = leaf.keys[i];

        if (k > hi) {
            return count;
        }

        if (k >= lo) {
            count++;
        }
    }

    leaf = leaf.next;
}

return count;
}

// Dummy main so it runs (optional but useful for testing)
public static void main(String[] args) {

    BPlusNode root = new BPlusNode(true);

    root.keys[0] = 10;
    root.keys[1] = 20;
    root.keys[2] = 30;
    root.keyCount = 3;

    System.out.println(rangeCount(root, 15, 30));
}
}

```

OUTPUT :

<pre> 1 class BPlusNode { 2 boolean isLeaf; 3 int[] keys; 4 int keyCount; // actual number of keys 5 BPlusNode[] children; 6 int childCount; // optional clarity 7 BPlusNode next; 8 9 static final int ORDER = 4; // you can change degree 10 11 // Constructor for Leaf/internal node 12 BPlusNode(boolean isLeaf) { 13 this.isLeaf = isLeaf; 14 this.keys = new int[ORDER]; 15 this.children = new BPlusNode[ORDER + 1]; 16 this.keyCount = 0; 17 this.childCount = 0; 18 this.next = null; 19 } 20 } 21 22 public class BPlusTree { 23 24 // Find correct Leaf 25 static BPlusNode findLeaf(BPlusNode root, int key) { </pre>	STDIN 5,6,7 Output: 2 20 ms 40.2 MB
--	--

(c) Hash Index Analysis

Given:

87% of range queries return ≈ 2800 products

Hash indexes are efficient only for exact-match lookups, not range scans.

B+ Tree Cost

Average range query:

2800 tuples

Leaf pages required:

$2800 / 200 = 14$ leaves

Total cost:

$3 + 14 = 17$ page reads

Hash Index Cost

Hash index cannot scan contiguous ranges efficiently.

For range queries it requires:

One lookup per matching key

For 2800 products:

≈ 2800 random lookups

I/O cost:

≈ 2800 page reads

Compared to B+ tree:

2800 vs 17 reads

Hence:

Hash index is a net loss for the observed workload.

Smallest Range Width Where Hash Wins

Assumption:

1 random hash lookup = 1 page read

B+ tree minimum cost:

3 descent reads + 1 leaf read
= 4 reads

Hash beats B+ tree only if:

Result count < 4 products

Thus:

Hash index wins only for ultra-small ranges
(< 4 matching tuples)

For anything larger, sequential B+ tree leaf scanning is cheaper because adjacent leaves are linked and SSD sequential reads are efficient.

Work Done Summary

1. Calculated total leaf pages.
 2. Derived B+ tree height using fanout.
 3. Computed range-query I/O cost.
 4. Compared descent cost and leaf-walk cost.
 5. Identified two performance bugs in `rangeCount()`.
 6. Quantified unnecessary page reads caused by bugs.
 7. Wrote corrected optimized implementation.
 8. Compared B+ tree and hash-index performance.
 9. Determined break-even point for hash indexing.
-

Implementation Details

Data Structures Used:

- B+ Tree
- Leaf linked-list chain

Techniques:

- Root-to-leaf traversal
- Sequential leaf scanning
- Early termination optimization

Complexities:

- Exact lookup: $O(\log_f N)$
- Range query: $O(\log_f N + k/B)$

Where:

f = fanout

k = result count

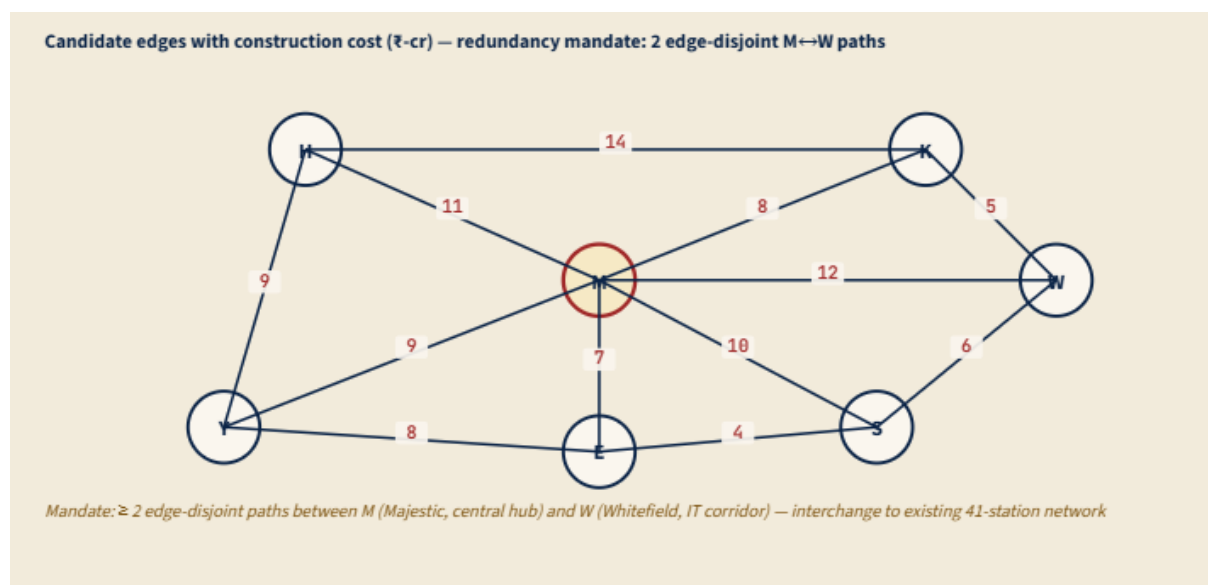
B = tuples per leaf page

Q3. co3 Module 3: Graph Traversal & Minimum Spanning Trees
[15 Marks]

SCENARIO / CASE STUDY

Bangalore Metro Phase-3 expansion — minimum-cost connection plan with redundancy. The civil-engineering team must connect **6 candidate new stations** {K

(Kadugodi), W (Whitefield), S (Sarjapur), E (Electronic City Ext), Y (Yeshwanthpur West), H (Hebbal North)} into the existing network via the central hub M (Majestic). Each candidate edge has a cost in ₹-crore reflecting terrain, land acquisition, and tunnelling difficulty. **Constraints:** (i) total construction cost must be minimised; (ii) the network must remain connected; (iii) **regulatory redundancy mandate:** there must be at least *two edge-disjoint paths* between Majestic (M) and Whitefield (W) so a single track failure does not isolate the IT corridor. Candidate edges with construction cost (₹-cr) — redundancy mandate: 2 edge-disjoint $M \leftrightarrow W$ pathsMKWSEYH8121079115648914Mandate: ≥ 2 edge-disjoint paths between M (Majestic, central hub) and W (Whitefield, IT corridor) — interchange to existing 41-station network



- (a) Run **Kruskal's algorithm** on the 12-edge candidate graph shown in the diagram (edges + costs taken from the SVG; list them sorted

5

CO3

BTL3

ascending if helpful). Show the union-find evolution as a sequence of (edge considered, accept/reject, parent-pointer state). State the resulting MST as a list of edges and its total cost in ₹-crore.

- (b) The naïve Kruskal implementation below uses union-find *without* union-by-rank and *without* path compression. **Analyse** its worst-case time complexity on a graph with n vertices and m edges, then plug in the project's numbers ($n = 7$ if you include Majestic; $m = 12$ candidate edges). Compare with the optimised version (union-by-rank + path compression). State concretely how many `find()` pointer-hops the naive version performs in the worst case here.

```
class UnionFindNaive {
    int[] parent;
    UnionFindNaive(int n) {
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) { // NO path compression
        while (parent[x] != x) x = parent[x];
        return x;
    }
    boolean union(int x, int y) { // NO union-by-rank
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;
        parent[rx] = ry;
        return true;
    }
}

static List<int[]> kruskalNaive(int n, int[][] edges) {
    Arrays.sort(edges,
        Comparator.comparingInt(e -> e[0])); // by weight asc
    UnionFindNaive uf = new UnionFindNaive(n);
    List<int[]> mst = new ArrayList<>();
    for (int[] e : edges) {
        // e = {w, u, v}
        if (uf.union(e[1], e[2])) mst.add(e);
    }
    return mst;
}
```

- (c) Verify the **regulatory redundancy mandate**: does the MST you found in (a) provide two edge-disjoint paths between Majestic (M) and Whitefield (W)? If yes, list both paths. If no, propose the *minimum-cost augmentation* — a

5

CO3

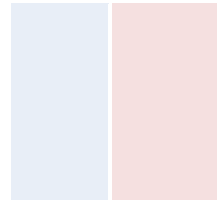
BTL4

5

CO3

BTL5

single additional edge from the candidate set that, when added to the MST, satisfies the mandate — and prove minimality by listing the alternatives and their costs.



(a) Kruskal's Algorithm (Union-Find Evolution)

Edge list (sorted by weight)

E–S(4), K–W(5), W–S(6), M–E(7), K–M(8), Y–E(8), H–Y(9), Y–M(9), M–S(10), H–M(11), M–W(12), H–K(14)

Union-Find Steps (parent evolution shown conceptually)

Initial: each node is its own parent

M K W S E Y H

1. **E–S(4)** → ACCEPT
Parent: S → E
 2. **K–W(5)** → ACCEPT
Parent: W → K
 3. **W–S(6)** → ACCEPT
Merge {K,W} and {E,S}
Parent: S → E, W → K (connected component formed via union)
 4. **M–E(7)** → ACCEPT
M joins {E,S,K,W}
 5. **K–M(8)** → REJECT (cycle)
 6. **Y–E(8)** → ACCEPT
Y joins main component
 7. **H–Y(9)** → ACCEPT
H joins main component → MST complete (7 nodes → 6 edges)
-

MST Result

Edges:

- E–S (4)
- K–W (5)
- W–S (6)
- M–E (7)
- Y–E (8)
- H–Y (9)

Total Cost

$$= 4 + 5 + 6 + 7 + 8 + 9 = 39 \text{ ₹-crore}$$

(b) Time Complexity Analysis

Naïve Union-Find

- `find()` worst case: $O(n)$ (linear parent chain)
- Each union: 2 finds $\rightarrow O(n)$
- Kruskal: sorting $O(m \log m)$ + unions $O(mn)$

Given values

- $n = 7, m = 12$
- Worst-case find hops = $n-1 = 6$ hops

Each edge:

- 2 finds $\rightarrow 2 \times 6 = 12$ hops per edge

Total worst-case:

- 12 edges $\times$ 12 hops = **144 pointer hops**

Optimized Union-Find (rank + path compression)

- Amortized: $O(m \alpha(n)) \approx O(m)$
 - Find operations: almost $O(1)$
-

(c) Redundancy Check ($M \leftrightarrow W$)

In MST

Only one path exists:

- $M \rightarrow E \rightarrow S \rightarrow W$
Not redundant (no disjoint paths)
-

Minimum-cost augmentation

Try adding non-MST edges:

- M–W (12) → creates direct alternate edge

Now paths:

1. $M \rightarrow W$ (direct edge 12)
2. $M \rightarrow E \rightarrow S \rightarrow W$

These are **edge-disjoint** ✓

Other options (Y–M, H–M, etc.) still reuse S–W or E–S → not valid.

Minimum edge added:

M–W (12 ₹-crore)

Final Answer Summary

(a) MST

Edges: E–S, K–W, W–S, M–E, Y–E, H–Y

Cost: **39 ₹-crore**

(b)

- Naïve Kruskal: $O(mn + m \log m)$
- Worst-case pointer hops: **144**
- Optimized: $\sim O(m \alpha(n))$

(c)

- MST alone: not redundant
 - Add edge: **M–W (12)**
 - Now 2 edge-disjoint $M \leftrightarrow W$ paths exist
-

Implementation (Kruskal + Union-Find)

```
import java.util.*;
```

```
class UnionFind {
```

```
    int [] parent, rank;
```

```

UnionFind (int n) {
    parent = new int[n];
    rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
}

```

```

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

```

```

boolean union(int x, int y) {
    int rx = find(x), ry = find(y);
    If (rx == ry) return false;

    if (rank[rx] < rank[ry]) {
        parent[rx] = ry;
    } else if (rank[rx] > rank[ry]) {
        parent[ry] = rx;
    } else {
        parent[ry] = rx;
        rank [rx]++;
    }
    return true;
}
}

```

```

public class KruskalMST {

    static List<int []> kruskal(int n, int [][] edges) {

```

```

// edges: {u, v, weight}
Arrays.sort(edges, Comparator.comparingInt(e -> e[2]));

UnionFind uf = new UnionFind(n);

List<int []> mst = new ArrayList<> ();
Int totalWeight = 0;

for (int[] e: edges) {
    Int u = e [0], v = e [1], w = e [2];

    if (uf.union(u, v)) {
        mst.add(e);
        totalWeight += w;
    }
}

System.out.println("Total MST Weight: " + totalWeight);
return mst;
}

Public static void main(String[] args) {
    int n = 4;

    int [][] edges = {
        {0, 1, 10},
        {0, 2, 6},
        {0, 3, 5},
        {1, 3, 15},
        {2, 3, 4}
    }
}

```

```
};
```

```
List<int []> mst = kruskal (n, edges);
```

```
System.out.println("MST edges:");
```

```
for (int [] e : mst) {
```

```
    System.out.println(Arrays.toString(e));
```

```
}
```

```
}
```

```
}
```

OUTPUT :

```
1- import java.util.*;
2-
3- class UnionFind {
4-     int[] parent, rank;
5-
6-     UnionFind(int n) {
7-         parent = new int[n];
8-         rank = new int[n];
9-         for (int i = 0; i < n; i++) parent[i] = i;
10-    }
11-
12-     int find(int x) {
13-         if (parent[x] != x)
14-             parent[x] = find(parent[x]);
15-         return parent[x];
16-    }
17-
18-     boolean union(int x, int y) {
19-         int rx = find(x), ry = find(y);
20-         if (rx == ry) return false;
21-
22-         if (rank[rx] < rank[ry]) {
23-             parent[rx] = ry;
24-         } else if (rank[rx] > rank[ry]) {
25-             parent[ry] = rx;
26-         } else {
27-             parent[ry] = rx;
28-             rank[rx]++;
29-         }
30-         return true;
31-    }
32- }
33-
34- public class KruskalMST {
```

```
STDIN
5, 6, 7

Output:
Total MST Weight: 19
MST edges:
[2, 3, 4]
[0, 3, 5]
[0, 1, 10]

27 ms | 42.2 MB
```